Adv Algs for ML: Assignment 1

pstoilescu

October 2026

1 Introduction

GitHub username: `FailedMath3U`. Repository name: `MLDS`. Link: `https://github.com/FailedMath3U/MLDS`. The code, tests and saved numerical results are included there; run instructions are at the end of this report.

For both games, the main plots use training seed 1668 and the final fixed-budget model. Evaluation uses 500 fresh episodes with sampled actions and no learning. Plots and evaluation statistics use undiscounted episode rewards, while training uses the prescribed discounted objective. Standard deviations below are population standard deviations. CartPole used CPU only; Pong used one GPU.

2 Value and Policy Iteration: Shuttle Problem

2.1 Model

We will model the prescribed shuttle decision problem as a Markov decision process. Thus, we must specify the states, actions, transition probabilities (conditional on actions), rewards and the discount factor. The states S are the number of passengers waiting at the station, i.e. one of the 201 elements of $\{0, \dots, 200\}$. The action set is the two element set given by either calling the shuttle or not. We denote an action by $a \in \{0, 1\}$ with 0 representing not calling a shuttle and 1 representing calling one (we're **assuming** at most one shuttle can be called per decision epoch). After we've chosen our action for stage t, suppose we are in state s_t, then $\min(15, s_t)$ people will leave the queue at the bus station if we call the shuttle, and A_t (i.i.d. uniform on $\{1, \dots, 5\}$) people will arrive. Here we will have to make an **assumption** to disambiguate the problem: we assume that the new arrivals come after the shuttle potentially picks up passengers. Thus the number will change by $A_t - \min(15a, s_t)$, before applying the 200 cap.

So we define the random variable of the next state to be

$$s_{t+1}(s_t, a) = \min\left(s_t - \min(15a, s_t) + A_t,\, 200\right).$$

Note that the transition probabilities are usually $1/5$ or 0, unless we're close to the 200 cap. More precisely, for any $s_t, s_{t+1} \in S$

$$P(s_{t+1} \mid s_t, a) = \frac{1}{5} \sum_{k=1}^{5} \mathbf{1}\left\{s_{t+1} = \min\left(s_t - \min(15a, s_t) + k,\, 200\right)\right\}.$$

Our rewards will be deterministic/non-random. We will **assume** that holding cost is charged for passengers remaining immediately after dispatch, before new arrivals. Given a state s and action a let $c(s, a)$ denote our cost. Then our reward will be $-c(s, a)$. For the sake of convenience we will mostly use costs for the rest of the section. The plots also show the reward value, which is the negative of the cost. So we will be talking about minimizing the cost rather than maximizing the reward (mathematically equivalent). Recall that the dispatch cost is $100a$ and the holding cost is $2\max(s - 15a, 0)$. Therefore

$$c(s, a) = 100a + 2\max(s - 15a, 0)$$

Recall that the discount factor has been specified to be $\gamma = 0.95$. We also assume overflow above 200 is discarded without an extra penalty, and that there is no separate shuttle-availability or travel-time constraint. Thus we have completely modeled our problem as a formal Markov decision process.

2.2 (a) Enumeration (Finite Horizon)

Given the requested plot and the 500 step limit, we assume that we're being asked to compute the optimal value at time 0 by enumerating the optimal values at all time steps using backward induction. This computes the optimal cost of the finite-horizon problem, up to numerical error. It approximates the infinite-horizon cost, and its policy may change depending on the time.

We start by having our final values be $J_{500}(s) = 0$, assuming no terminal penalty. Then we iteratively apply the Bellman rule, i.e. that the optimal cost is recursively given by

$$J_t(s) = \min_a \left[c(s,a) + \gamma \mathbb{E}[J_{t+1}(s_{t+1}(s,a))] \right],$$

to compute all the expected costs backwards. By the definition of s_{t+1} our program computes the expectation as the mean of 5 values. This avoids the exponential complexity of computing the cost for all sequences of actions. So for each time step we compute the optimal value for all states before computing the previous timestep. Thus we end up with the optimal cost at the initial time for all possible states. Thus to compute the optimal cost for a single state is roughly as computationally intensive as computing it for all states. So the following plot is useful for validation even if you were only interested in a single optimal value.

$$J_t(s) = \min_\pi \mathbb{E}_\pi \left[\sum_{u=t}^{499} \gamma^{u-t} c(s_u, a_u) \,\middle|\, s_t = s \right].$$

Here $J_t(s)$ is the minimum expected remaining discounted cost; π is the decision rule and $\mathbb{E}_\pi$ averages the random arrivals. Define the reward value by $V_t(s) = -J_t(s)$.

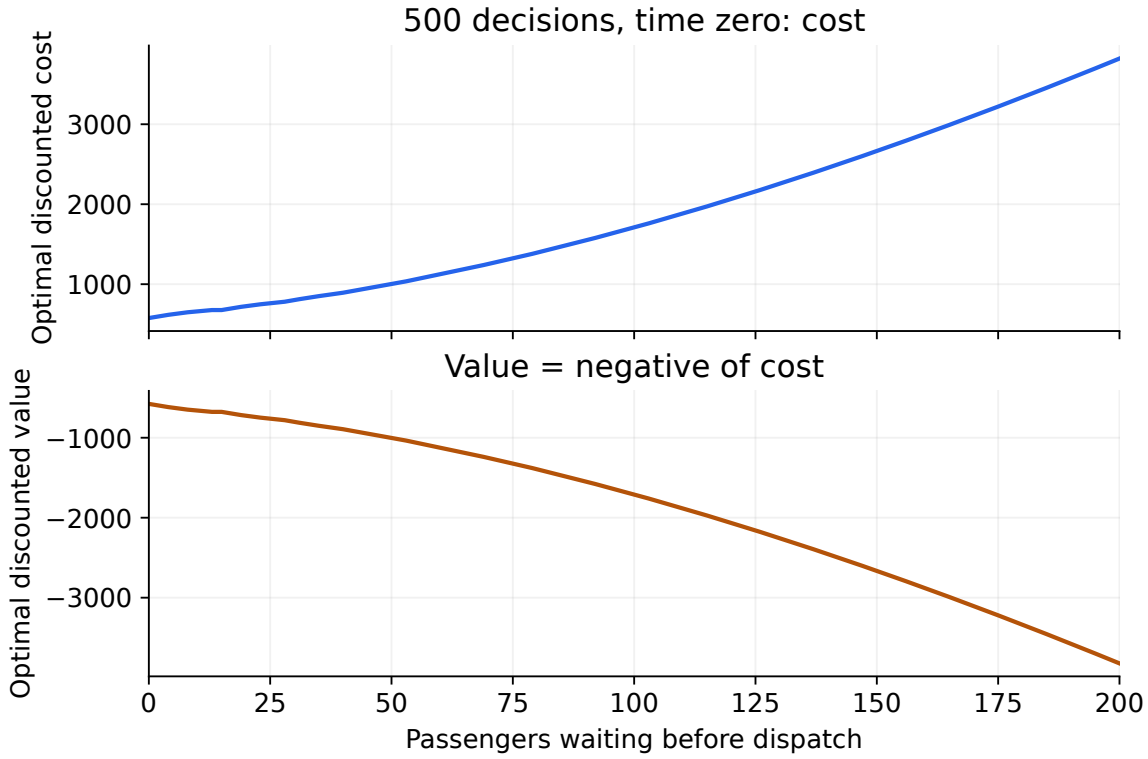


Figure 1: Enumeration, 500 decisions: optimal time-zero cost $J_0(s)$ (top) and reward value $V_0(s) = -J_0(s)$ (bottom).

The general shape (increasing slope) is what we would expect since we can only send one shuttle at a time; a larger queue means more people waiting and more time needed to clear it, although discounting means the cost need not be exactly quadratic. The cost is also positive even for 0 people at the start, which

is what we should expect random people will almost certainly pop up, and we'll incur a cost either neglecting them or calling a shuttle. Thus, we've passed the most basic validity (sanity) checks for our algorithm.

2.3 (b) Value Iteration (Infinite Horizon)

Now we compute the optimal cost of the MDP using the time invariance of it. We use Bellman updates to iteratively converge towards the optimal value function. Superscripts will denote iterations. Initialize $J^0(s) = 0$ for all $s \in S$. Then if $\boldsymbol{s}(s, a)$ is the random variable for the next state

$$J^{n+1}(s) = \min_a \left[c(s, a) + \gamma \mathbb{E}[J^n(\boldsymbol{s}(s, a))] \right].$$

Note that as $\boldsymbol{s}(s, a)$ is described above, the program computes the expectation as the mean of 5 values. Because the Bellman update is a contraction with factor $\gamma < 1$, it converges. We terminate when another Bellman update would change the current vector by at most 10^{-10} in every state (the Bellman residual). In this case, empirically we need 518 updates. The plot for value iteration is almost identical to the plot for enumeration, which is what we should expect since 500 is large relative to $\gamma = 0.95$. This consistency supports the numerical results, although the methods share the same model.

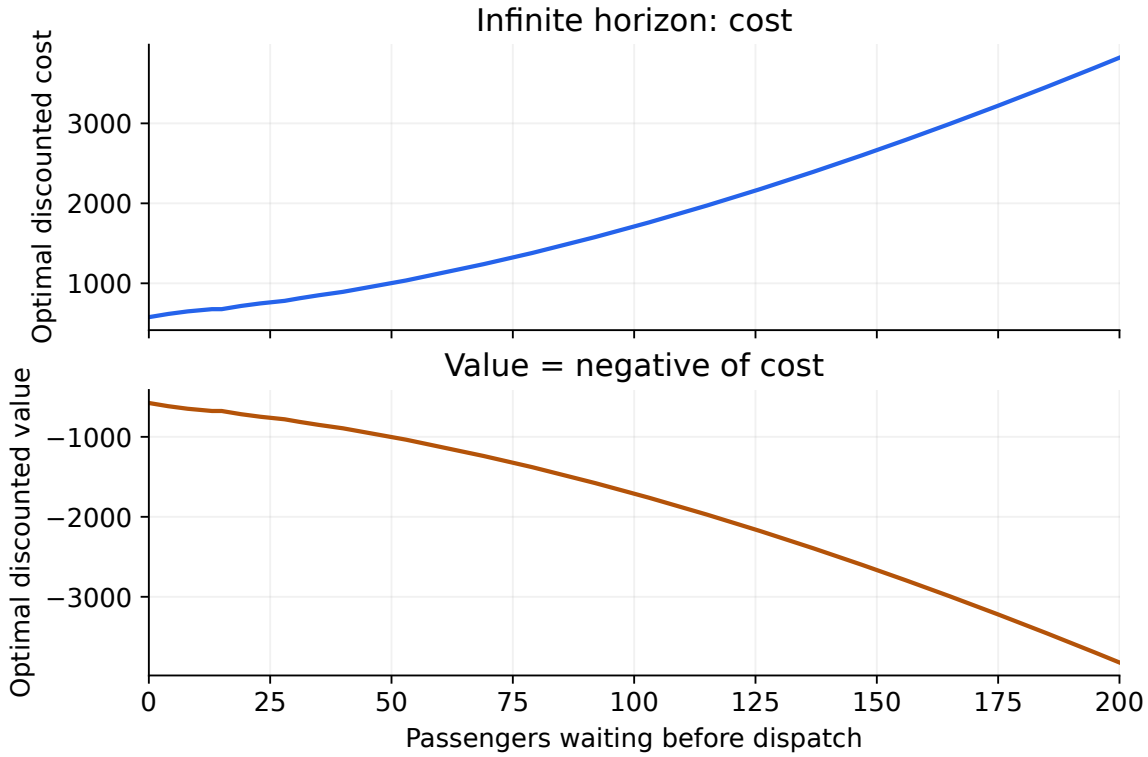


Figure 2: Value iteration: optimal infinite-horizon cost $J^*(s)$ (top) and reward value $V^*(s) = -J^*(s)$ (bottom).

2.4 (c) Policy Iteration (Infinite Horizon)

Policy iteration is an alternating process. Given a policy π (in this case a function from states to actions), its cost satisfies

$$J_\pi(s) = c(s, \pi(s)) + \gamma \mathbb{E}[J_\pi(\boldsymbol{s}(s, \pi(s)))].$$

It's actually implemented using the J vector (for all states) and the transition probability matrix P_π. We solve

$$(I - \gamma P_\pi) J_\pi = c_\pi,$$

where I is the identity matrix, $P_\pi(s, s') = P(s' \mid s, \pi(s))$ and $c_\pi(s) = c(s, \pi(s))$. Thus our code uses a linear solve for policy evaluation, rather than an inner iteration with a tolerance. Then we see if we can greedily improve the policy by defining a new policy

$$\pi_{\text{new}}(s) = \arg\min_a \left[c(s, a) + \gamma \mathbb{E}[J_\pi(\boldsymbol{s}(s, a))]\right].$$

It is guaranteed that this new policy is at least as low cost as π. We can then evaluate the new policy with the same linear solve, and alternate back into policy improvement. Repeating this, we are guaranteed to converge to the optimal policy. Since the policy is in this case necessarily discrete, we are guaranteed to arrive at the policy. This is when we get the same policy back from policy improvement. We can thus plot the optimal policy by plotting whether it'll dispatch or not at a given number of customers waiting. This took six evaluation/improvement rounds, starting from always waiting, including the final stability check. Ties are resolved by waiting.

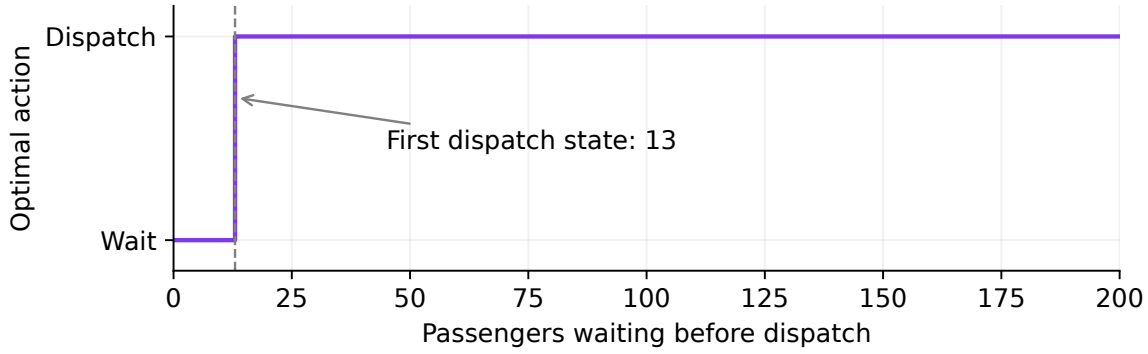


Figure 3: Policy iteration: wait at queues 0–12 and dispatch at queues 13–200.

The computed optimal policy dispatches whenever we're above a certain threshold and otherwise waits. No threshold was imposed on the search. If the cost of waiting times the number of waiters is sufficiently high relative to γ, it might even make sense to order a shuttle when you have less than 15. Thus the threshold has face validity. More quantitatively, at queue 12 the costs of waiting and dispatching, including optimal continuation, are 672.892 and 676.670; at 13 they are 680.770 and 676.670. These comparisons explain the threshold of 13.

2.5 Edge Cases

We prevent partially filled shuttles from causing underflow, i.e. negative customer counts by using $\max(s - 15a, 0)$.

Because we can't go above 200, some implementations may have transition probabilities not sum to 1. We handled this via our more complicated formula which is guaranteed to sum to 1.

We made sure that $J_{500} = 0$. There is no decision at $t = 500$, preventing ghost continuation costs. At $t = 499$, waiting is optimal in every state: dispatch costs 100 but can save at most $2 \cdot 15 = 30$ immediately, with no future benefit or terminal penalty. We checked this prediction against the saved final-period policy.

2.6 Validation and Correctness Tests

Value iteration and policy iteration selected the same actions for all 201 states. The differences of the values were at most 1.964×10^{-9}. The policy-iteration Bellman residual was 9.095×10^{-13}, bounding value error by residual$/(1 - \gamma) = 1.819 \times 10^{-11}$, subject to rounding. The minimum absolute difference between the two action costs across states was 3.778, meaning that numerical errors are unlikely to change the optimal action. Solver agreement is useful, but their shared model could still contain a common error.

We can also compare the results of the 500 epoch enumeration with the infinite horizon values. The maximum difference was 5×10^{-9}. The largest possible cost per timestep is 470, so a theoretical upper

bound on the difference between the 500 horizon and infinite timestep is

$$470 \sum_{t=500}^{\infty} \gamma^t < 7 \times 10^{-8}$$

But the observed difference is 5×10^{-9}, which is consistent with the above. This is also reflected by the visual impression that our two plots are virtually identical.

We have also checked that the queue never becomes negative or above 200. We also checked that dispatching with at most 15 leaves 0 before arrivals and that dispatching with 16 leaves 1. We checked that waiting at 200 still leads to 200 in the queue. We've checked that at zero passengers, our policy prefers to wait rather than dispatch. An exhaustive audit of all 2,010 state/action/arrival combinations also matched scalar transitions and costs, with every transition-probability row summing to 1.

For independent predictions, setting holding cost to zero gave zero optimal cost and always waiting; setting $\gamma = 0$ gave $J(s) = 2s$ and always waiting. A separately coded exhaustive decision/arrival tree with seven states and four decisions matched backward induction exactly.

2.7 Validation With an Additional 3 months

We would vary arrival distributions, dispatch and holding costs, and the queue cap to see how the threshold changes. We would also compare a separately implemented linear-programming solution, and test overflow penalties and terminal costs. Adding correlated arrivals or shuttle travel times would test whether the original policy remains useful when its modeling assumptions change.

3 Policy Gradient

REINFORCE Given a differentiably parametrized family of policies $\{\pi_\theta\}_\theta$, for a MDP, we can optimize the parameters to maximize the expected value under that policy. One such class of methods are called policy gradient methods; they're based on stochastic gradient optimization of the hard-to-compute return function. The most basic such algorithm is called REINFORCE. Given a θ we want to nudge it to improve expected value under the parametrized policy. We will find an easy to compute unbiased estimator of the gradient of expected discounted return under π_θ given actions and states and do stochastic gradient optimization using that. We sample a sequence of states and actions S_t and A_t (an episode), following the policy π_θ, we call the future value from timestep t of such a sequence G_t. Thus G_t is an unbiased estimator of $Q_{\pi_\theta}(S_t, A_t)$. Then we use terms $\gamma^t G_t \nabla_\theta \log \pi_\theta(A_t \mid S_t)$ in our gradient. The log comes from the identity $\nabla_\theta \pi_\theta = \pi_\theta \nabla_\theta \log \pi_\theta$. We could use standard stochastic gradient ascent, but the assignment specifies the Adam optimizer. Hence given a stepsize $\alpha > 0$, we sum the discounted gradient terms within each episode, average over the batch of episodes, and use the result for one Adam update. The objective is $\mathbb{E}[\sum_{t=0}^{T-1} \gamma^t r_t]$, where T is the episode length and r_t its reward after decision t. Our code uses a fixed gradient scale of 0.05 for CartPole and 1 for Pong; it does not standardize returns. Gradient-norm caps are 10 and 5 respectively, so the clipped estimator need not remain unbiased.

3.1 CartPole Model

The cartpole is modeled as a continuous space, discrete action, and effectively discrete time (0.02 s timesteps). The state space is the position of the cart, velocity of the cart, angle of the pole and rate of change of the angle of the pole, with failure when the angle exceeds 12° or the cart leaves $[-2.4, 2.4]$. Episodes are also capped at 500 steps. The action is to apply a force of 10 Newtons either left or right. The transition rules are entirely deterministic given state and action, a numerical application of Newton's laws. The initial four components are sampled uniformly from $(-0.05, 0.05)$, with a distinct recorded seed per episode. The reward is +1 for every transition, including the failure transition. The discount factor is the specified $\gamma = 0.95$.

3.2 CartPole Policy

We will use a parametrized policy given by a neural network (MLP) that directly takes the state's 4 parameters as its input layer. There is one hidden layer with 64 units and tanh activation functions. Then the

final layer gives logits, which are passed through softmax to determine the probability of accelerating left or right.

3.3 CartPole REINFORCE results

Because each episode is randomly sampled from the policy, there can be variance between episodes from even the same policy, let alone changing policies per update. Thus we needed plot the 100 average return, to reduce the noise and have any trend apparent. But we still have to wonder if this is sufficient. So I plotted the 100 average of episode returns with an earlier policy frozen for 2,000 fresh episodes. Its averages ranged from 430.11 to 472.29, while continued training fell to 74.64. Frozen checkpoints before and during the dip scored 453.108 and 83.114 over 500 fresh paired episodes, with a paired 95% bootstrap interval of $[-377.430, -362.038]$ for the change. Thus the selected dip reflects a policy change beyond fixed-policy episode noise. The checkpoints were selected after looking at the curves; this does not identify the cause of the bad updates. As we can see in the plots, the training initially improves but is very unstable. We've experimented with smaller step sizes, but the final performance was not consistently better. Across three seeds, the basic method's mean final evaluation scores were 418.570, 163.178 and 247.102 for rates 0.01, 0.005 and 0.002 respectively. A good baseline can reduce gradient variance, but does not guarantee steadier training gains.

We trained for 8,000 episodes with Adam rate 0.01 and batches of 32 complete episodes. The primary final policy scored 500.000 ± 0.000 over 500 evaluations; all reached the limit without physical failure.

3.4 CartPole Edge Cases

We tested that the game really ends when the angle exceeds its allowed range or the cart leaves its allowed range. The final transition still received a +1 reward, which we incorporated into the return.

The timelimit is 500 and we stopped the episode there if it had not already failed. We checked time-limit truncation separately from physical termination. Either flag ends our capped episode, with zero continuation. A failure on the last allowed step can set both flags; we tested that case too.

We updated by using the gradient for 32 episodes. Within each episode, decision terms are weighted by $\gamma^t G_t$ (and the fixed scale), summed, and then averaged over 32 episodes. Dividing each episode contribution by its own length would change this estimator. Dividing the whole batch by its total state count would instead rescale that batch, with a scale depending on episode lengths.

3.5 CartPole Validation and Correctness Checks

An untrained neural policy scored 20.926 ± 11.467 and uniform random actions scored 22.782 ± 12.058, each over 500 episodes. These controls support improvement beyond initialization. For an analytic check, a two-step Bernoulli-action task with reward equal to its action has objective $(1 + \gamma)p$, where p is the probability of action 1. Enumerating all four paths gave gradient 0.471188756 versus analytic derivative 0.471188748. Returns $(1, 1, 1)$ also matched $(2.8525, 1.95, 1)$.

Controlled episodes of lengths 1, 1 and 3 exercised the actual batched collector. The one-step return was 1 and the combined gradient matched the average of separate episode gradients to 1.863×10^{-9}. A saved ten-step trace records states, probabilities, actions and rewards; at step 3, left was sampled even though right had probability 0.7711, as can happen under a stochastic policy.

3.6 CartPole 3 Additional Months

We would test more training seeds and initial positions, angles and velocities outside the narrow reset region. We would vary batch size and baseline-fitting frequency alongside learning rate, and measure critic error and gradient variance throughout training to investigate the dips rather than only extending training.

3.7 Pong Model

We're preprocessing the pong game. We use a cropped and downsampled 80×80 grid of binary pixels. Furthermore to capture the direction of the ball, the model uses the difference between consecutive images.

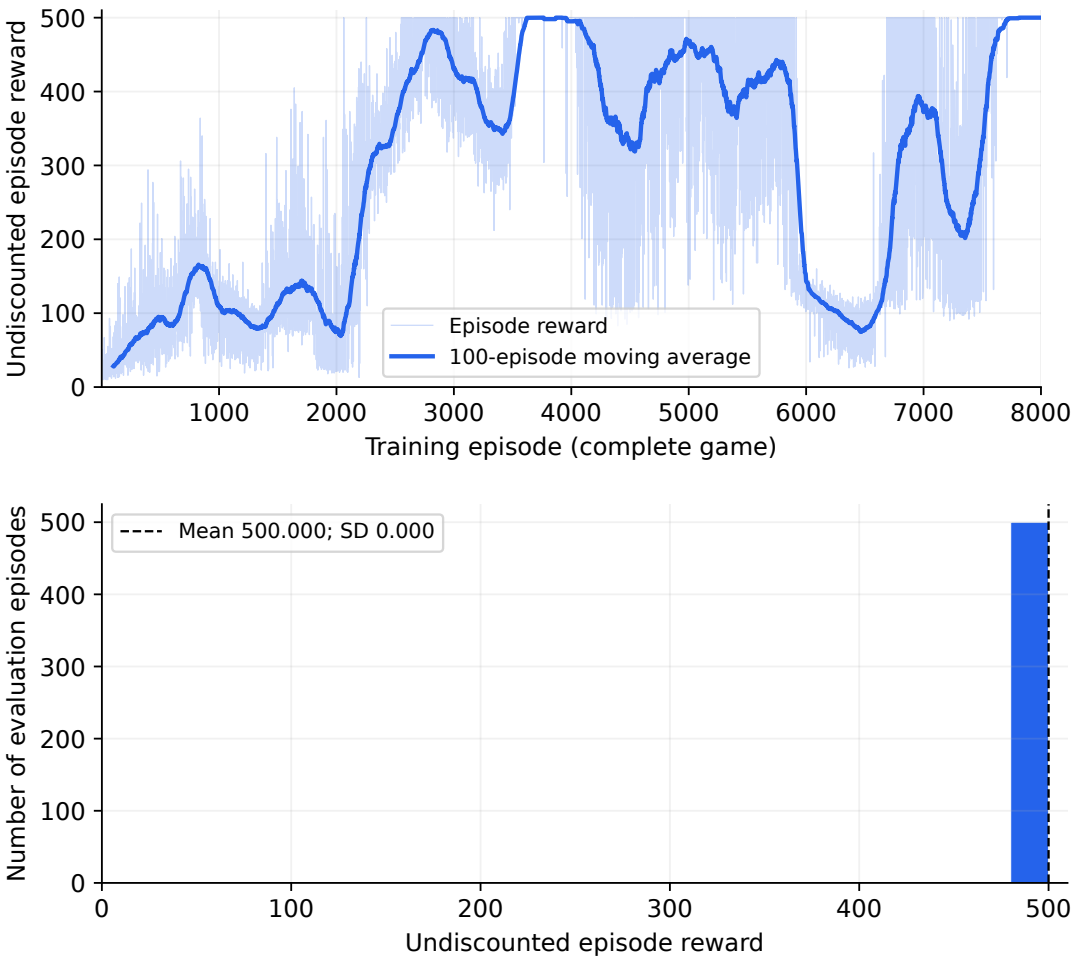


Figure 4: CartPole REINFORCE, seed 1668. Top: every training episode reward and its trailing 100-episode average. Bottom: 500 stochastic final-policy evaluation rewards; mean 500.000, population standard deviation 0.000.

So our observation implicitly contains cues to the direction the ball is moving. Although if the paddle is stationary (possible at the boundary), the difference is 0 so the paddle diapears. This can hide its position and may hinder our model; we have not established that the policy learns to avoid the problem. The two possible actions are ALE actions 2 and 3 (RIGHT and LEFT), which move the paddle vertically in opposite directions. They are directional commands, not a verified fixed pixel displacement. Because of the downsampling and the potentially hidden puck, it is a partially observable Markov decision process, not a strict MDP. The state transition probabilities are determined by the pong-v5 game. We get a reward of 1 when we score and -1 when the opponent scores. Note that our implementation is model-free: we only care about our observed states (partial information), action and rewards. We do not directly use or compute any transition probabilities; the policy still computes action probabilities. We use the specified discount factor of $\gamma = 0.99$. The game ends whenever a player reaches a score of 21 or the time limit. We stop return accumulation at either episode-ending flag. Also one timestep for our model is 4 timesteps for the Pong-v5 emulator.

3.8 Pong Policy

Our parametized policy will be a fully-connected feedforward neural network with layers of size $80^2 = 6400$, 200 and then 2. We use ReLU as our activation functions for computing the middle layer activations. Then

we use a linear layer applied to those to get 2 logits which are fed into softmax to get our final probabilities for our two actions.

3.9 Pong REINFORCE results

We used a learning rate of $\alpha = 0.001$ and the Adam optimizer to train our policy neural network. We chose this lower rate because, on identical game data, one update changed the action distribution less than rate 0.01 (mean KL divergence 1.996×10^{-6} versus 3.423×10^{-4}). This is a step-size check, not proof of better long-run stability. We trained for 20,000 full games, updating after batches of 8. The final score was -18.610 ± 1.681 over 500 games, versus -20.250 ± 0.935 for a 32-game untrained reference. All 500 final games were losses and none was truncated, so the policy did not learn to win.

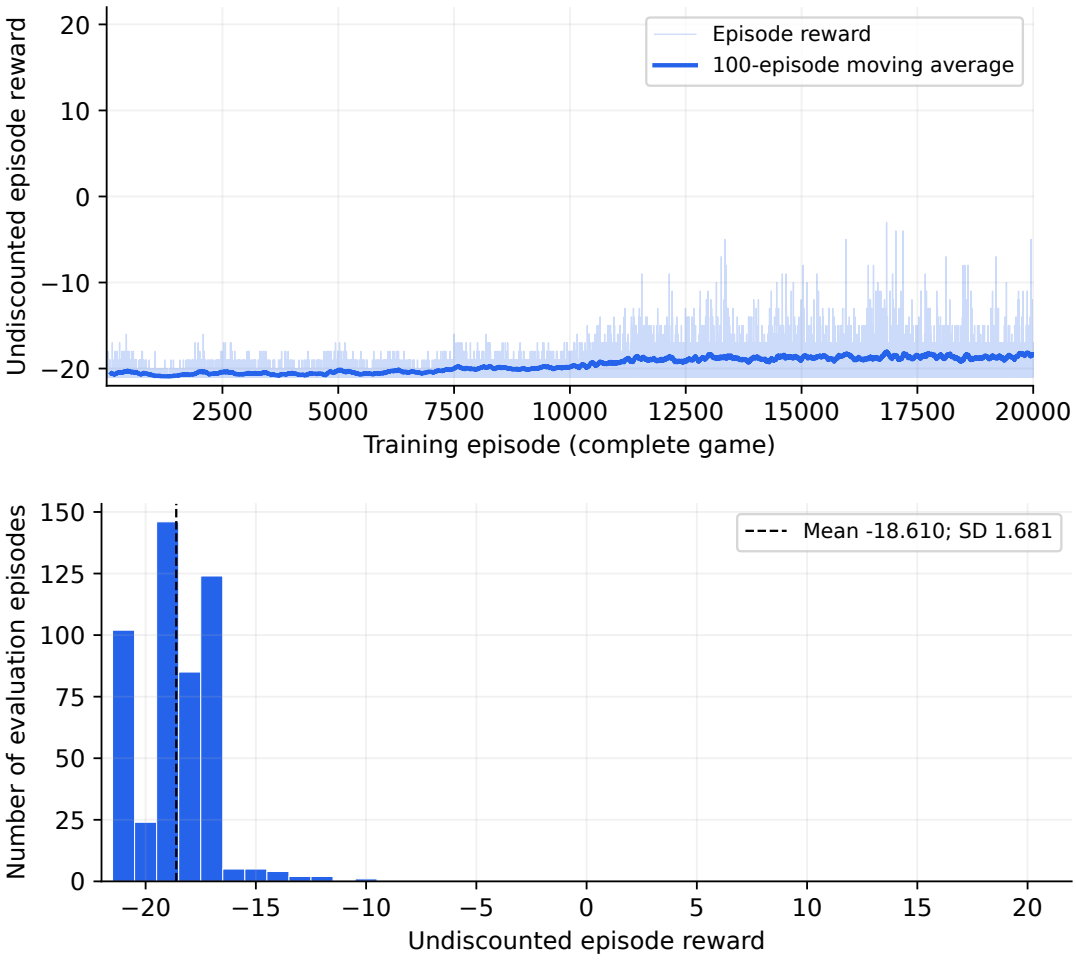


Figure 5: Pong REINFORCE, seed 1668. Top: every training episode reward and its trailing 100-episode average. Bottom: 500 stochastic final-policy evaluation rewards; mean -18.610, population standard deviation 1.681.

3.10 Pong Edge Cases, Validation and Correctness Checks

Some student implementations may end an epsisode whenever someone scores, but we made sure to calculate G_t using the full episode until one player gets 21 points. For example we manually tested that the rewards $(0, 1, 0, -1)$ give G_t values of $(0.019701, 0.0199, -0.99, -1)$. So our program accords with our mathematical calculation to many significant figures.

A game can end through termination or a time limit. We tested that we don't continue to compute returns past the game end in either situation. A real complete-game test ended after 1008 decisions with final reward and return both -1; a forced one-step limit truncated with final reward and return both 0. Neither collector stepped the environment after ending.

The first frame has no precessor, potentially causing a problem when we want to feed in the difference into our neural network. We approach this by feeding in the image with all zeros into our neural network for the first move. This means the first action has no motion information; later differences use only frames from the same game.

Another edgecase is that we need negative values in our image vectors, so we made sure to use signed arithmetic, since otherwise there may have been overflow errors.

The environment includes a FIRE command, but here our bot couldn't play fire. This did not prevent play: eight preflight games using only actions 2 and 3 each produced 21–22 scoring events and ended naturally. This does not show that FIRE has no effect on unrestricted gameplay.

If the paddle tries to move into the boundary it becomes stationary, so it disappears from the difference image. Boundary commands are not directly penalized; a negative reward occurs only when the opponent scores. Thus we cannot assume rewards automatically solve the missing-position problem.

Pong-v5 has sticky actions that can repeat the previous action regardless of the chosen action. We retained this behavior to match v5. The repeat probability is 0.25 per emulator frame, so within four frames there can be zero, one or more repetitions; it is not guaranteed that exactly one requested action is ignored. Identical reset seeds and fixed action sequences reproduced frame/reward hashes. This tests reproducibility, not that stickiness has a small effect.

3.11 Pong 3 Additional Months

We would compare frame differences with frame stacks and a small convolutional policy, retaining v5 and actions 2/3. We would measure how much of the gradient comes from late-game rewards, since $0.99^{1000} \approx 4.32 \times 10^{-5}$, and compare learning rates, batch sizes and clipping on separate validation seeds. Multiple training seeds would be needed to distinguish a robust improvement from a favorable run.

4 Policy Gradient with Baseline

A baseline is just an amount we subtract off of G_t that does not depend on the current action, given the information before that action. This doesn't change the expectation of the unclipped gradient estimator but changes its variance. The right choice of baseline will reduce the variance. We can pick a constant baseline or predict a baseline based on the state and time with e.g. a neural network. For CartPole we will use a neural network and for Pong a constant.

4.1 CartPole Baseline

We use a new neural network with input the state and the time (divided by 500). We have a 64 node hidden layer with tanh activation functions. Then we have a single output node that tries to predict G_t. We update our actor model after completing our batch of 32 episodes, using detached baseline predictions from its existing weights. Afterward we train the separate baseline with 5 Adam steps at learning rate 0.01 to minimize MSE against the observed G_ts. Thus current outcomes are used to fit predictions for later batches.

The plot still has large changes, and the primary final model scored 260.308 ± 57.013, worse than the basic model. Four of the 500 evaluations reached 500; 496 failed physically. Theory does not guarantee a better final policy. Across three seeds, the average final scores were 323.588 with the baseline and 418.570 without it.

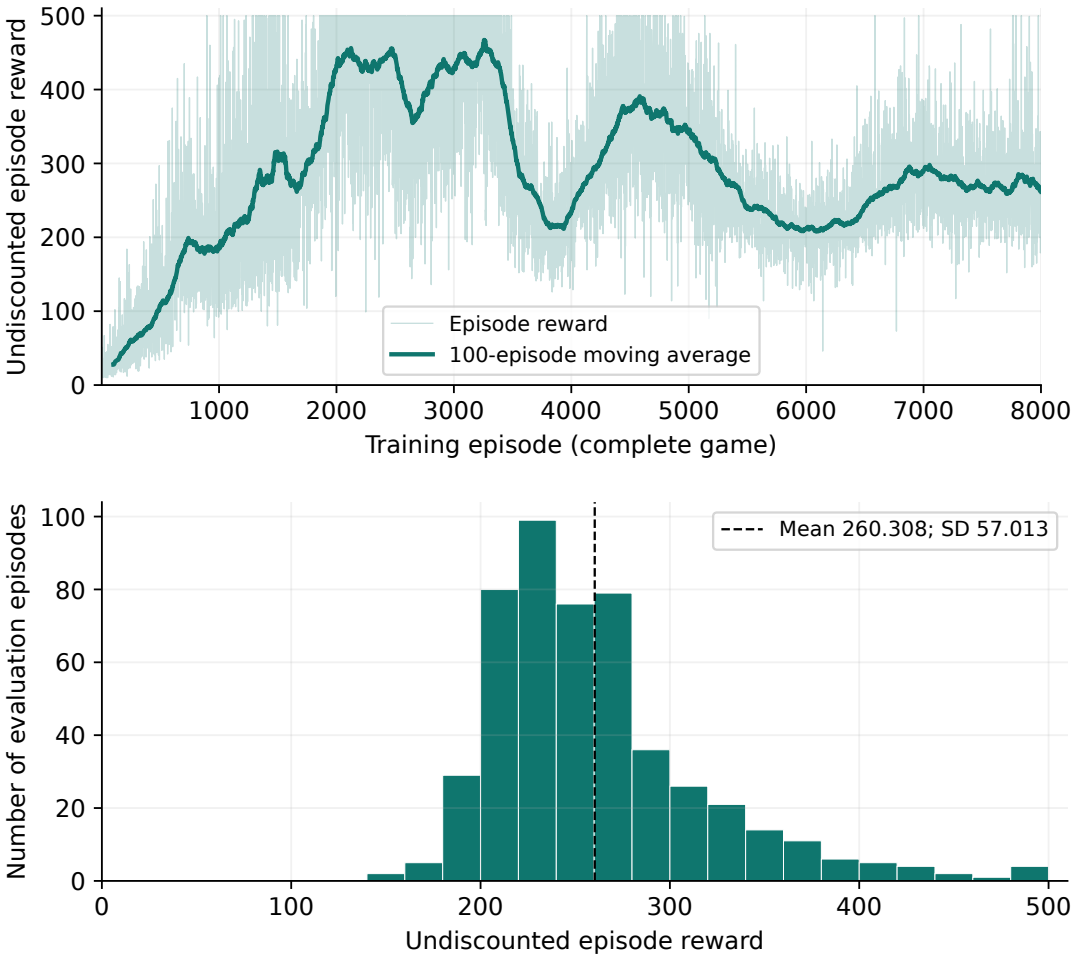


Figure 6: CartPole with a learned baseline, seed 1668. Top: every training episode reward and its trailing 100-episode average. Bottom: 500 stochastic final-policy evaluation rewards; mean 260.308, population standard deviation 57.013.

4.2 Pong Baseline

Here we use a scalar exponential moving average. Batch k uses the existing b_k, starting at $b_0 = 0$. After the actor update, we use this batch's returns to compute

$$\bar{G}_k = \frac{\sum_{i,t} \gamma^t G_t^{(i)}}{\sum_{i,t} \gamma^t}, \qquad b_{k+1} = 0.95 b_k + 0.05 \bar{G}_k.$$

The index i runs over all eight games, and t is the decision within each game. Thus it is a discount-weighted average of reward-to-go over the batch, not a second return calculation or a value fitted separately for each state.

The plot is fairly similar to the one without a baseline. The final result was -18.446 ± 1.851 over 500 games, only 0.164 above the basic method; all were losses. This does not establish a benefit across training seeds or lower gradient variance.

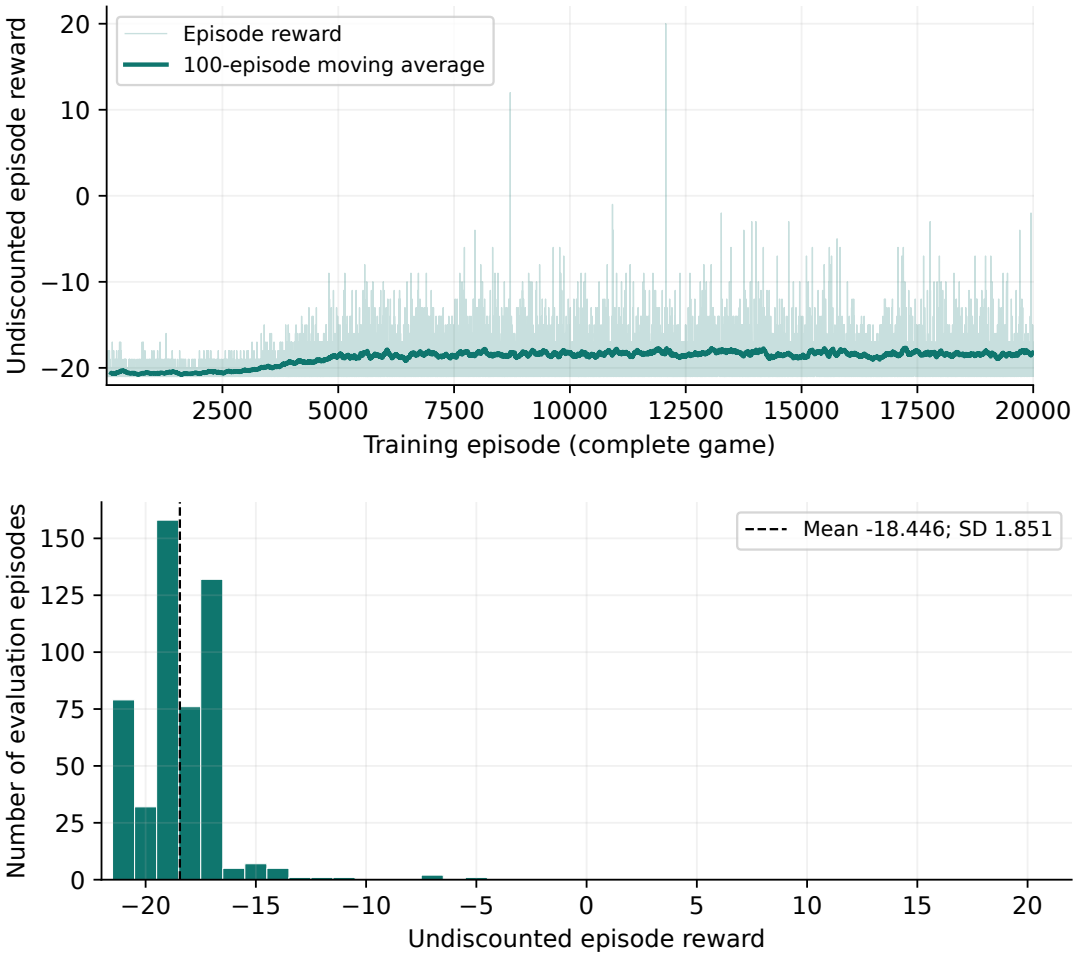


Figure 7: Pong with a constant EMA baseline, seed 1668. Top: every training episode reward and its trailing 100-episode average. Bottom: 500 stochastic final-policy evaluation rewards; mean -18.446, population standard deviation 1.851.

4.3 Edge Cases, Correctness Checks and Validation

If the rewards are all 0, then the target returns are 0 and a fitted baseline should approach 0. We tested this in both cases. Fitting the CartPole baseline for 400 Adam steps on 64 modified episodes gave held-out MSE 1.31×10^{-6} on 64 separate episodes. Pong's EMA stayed exactly 0 when initialized at 0; from 2 it decayed to 0.000546 after 160 zero-return updates, matching $2(0.95)^{160}$. The latter is a fixed-data baseline-mechanics experiment, not a new on-policy performance run.

Setting all the baselines to 0 recovers REINFORCE. On identical data, the maximum gradient difference was exactly 0 in both implementations. Zero returns with zero baseline also gave gradient norm 0.

We made sure to fit our baseline after updating the policy, so its predictions for the current actor update use weights from earlier data. Fitting on the current outcomes first can introduce dependence on the current actions, even if actions are not explicit network inputs.

If an episode lasts only one decision and every action receives reward 1, then all returns are 1. With an exact baseline of 1, both actor implementations gave gradient norm 0. Fitting the CartPole predictor on one-step episodes gave held-out mean 1.000019 and MSE 6.54×10^{-9}; Pong's EMA approached 0.999727 after 160 updates from 0. Finite fitting is approximate, so exact cancellation is checked separately with the baseline fixed to 1. Without a baseline, individual sampled gradients can be nonzero, although their expectation is still zero because every action has the same payoff. For longer episodes with reward 1 at every

decision, returns exceed 1, so the target baseline need not be 1.

Finally, we checked whether baselines actually reduced gradient variance. At a frozen initial CartPole actor, fitting a separate baseline on 512 episodes and testing on 512 different paired episodes reduced summed gradient-component variance from 5.338262 to 0.842421 (84.22%). This used 150 fitting steps, rather than the five per training batch. Pong's independently fitted initial constant reduced variance from 369.320 to 20.763 on 16 held-out games, but the actual EMA's later variance ratios were 0.873, 1.017, 0.924 and 1.011. Thus variance reduction did not persist at every checkpoint; these probes measure unclipped gradients, and 16 games gives noisy estimates.

4.4 3 More Months

We would compare constant, state-dependent and time-dependent baselines on identical frozen policies, using separate fitting data and more held-out episodes. We would track baseline error and gradient variance throughout training, with uncertainty intervals, and separate the effect of clipping from baseline subtraction. This would test when lower gradient variance actually produces better control.

5 Running the Code

Paths below are relative to `assignment1/` in the repository. Use a fresh copy for training to preserve saved runs, and install each project's `requirements.txt`. Shuttle runs with `python shuttle.py`; its separate `shuttle_plots.ipynb` regenerates plots. For CartPole:

```
cd mlds_assignment1_cartpole
python cartpole.py suite --episodes 8000 --lr 0.01
python validate.py
python validate_task_edges.py
python instability_experiment.py
python plot_instability_experiment.py
```

These run the six training/evaluation runs, numerical and environment checks, and exact replay/frozen-policy comparison. Evidence is saved in `runs/`, `validation/`, `task_validation/` and `instability_experiment/`. The lower-rate folders provide `run.sh`.

For Pong, activate the deepdish Pong environment and work in a fresh copy of `mlds_assignment1_pong`. Set a single GPU UUID consistently in `config.json`, `supervise.sh` and `CUDA_VISIBLE_DEVICES`; the supervisor also specifies the Python executable. Then run:

```
python run_pong.py preflight
nohup setsid bash supervise.sh > nohup.log 2>&1 < /dev/null &
```

Preflight checks returns, analytic gradients, image processing, restricted actions, seeded replay and checkpoint recovery. Training adds variance probes. Results are in `preflight/`, `runs/` and `diagnostics/`. Existing checkpoints resume instead of starting over.

The additional tests claimed in this draft are in `mlds_assignment1_submission/`. With PyTorch, Gymnasium and ALE installed, run:

```
python validate_report_claims.py
python validate_shuttle_and_images.py
python build_assets.py
tectonic ASSIGNMENT1_REPORT.tex
```

The first tests zero/unit-return baseline fitting, gradient cancellation, zero-baseline equivalence, and real Pong termination/truncation. The second checks every shuttle transition and signed/stationary image differences. Both save numerical evidence under `validation/` without modifying the original training files. The asset builder copies saved game plots and generates both cost and negative-cost shuttle plots.